

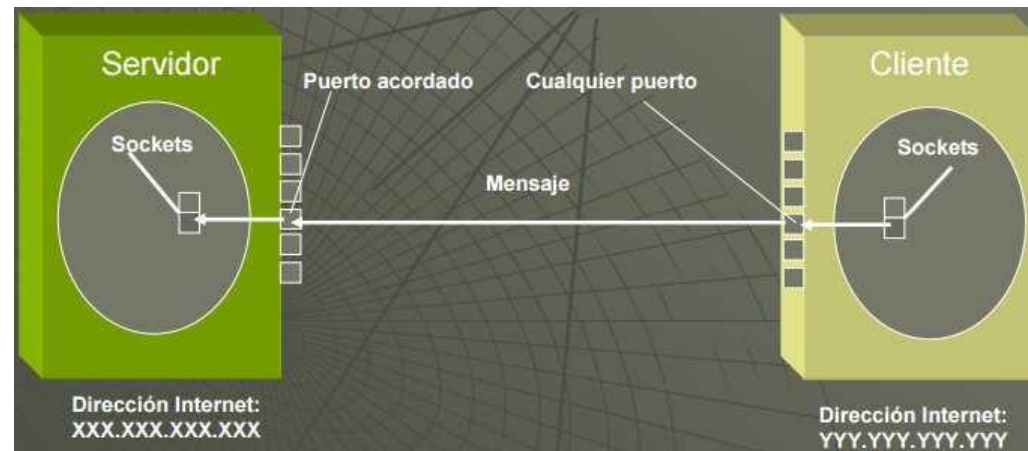
# Programación de Comunicaciones en Red

SOCKETS

### 1.- SOCKETS

Los sockets son un sistema de comunicación entre procesos de diferentes máquinas de una red. Más exactamente, un socket es un punto de comunicación por el cual un proceso puede emitir o recibir información.

Cada socket tiene un puerto asociado, el proceso cliente debe conocer el puerto y la IP del proceso servidor. El proceso cliente podrá enviar el mensaje por el puerto que quiera.



### 2.- Funcionamiento genérico de un SOCKETS

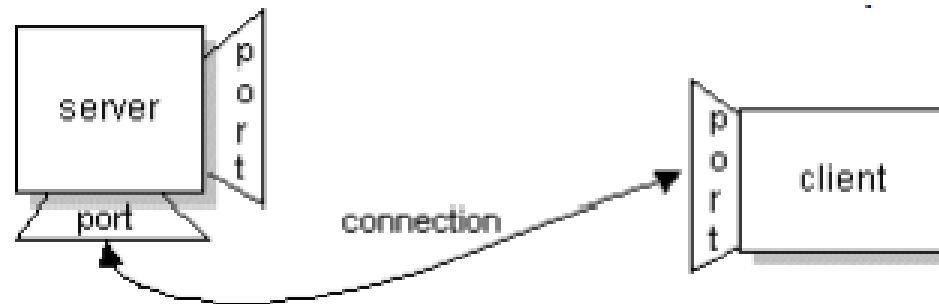
Normalmente, un servidor se ejecuta sobre una máquina específica y tiene un socket que responde en un puerto específico. El servidor únicamente espera, escuchando a través del socket a que un cliente haga una petición.

En el lado del cliente: el cliente conoce el nombre de host de la máquina en la cual el servidor se encuentra ejecutando y el número de puerto en el cual el servidor está conectado.

Para realizar una petición de conexión , el cliente intenta encontrar al servidor en la máquina servidora en el puerto especificado. Si todo va bien, el servidor acepta la conexión.



Si todo va bien, el servidor acepta la conexión. Además de aceptar, el servidor obtiene un nuevo socket sobre un puerto diferente. Esto se debe a que necesita un nuevo socket (y, en consecuencia, un número de puerto diferente) para seguir atendiendo al socket original para peticiones de conexión mientras atiende las necesidades del cliente que se conectó.



Por la parte del cliente, si la conexión es aceptada, un socket se crea de forma satisfactoria y puede usarlo para comunicarse con el servidor. Es importante darse cuenta que el socket en el cliente no está utilizando el número de puerto usado para realizar la petición al servidor. En lugar de éste, el cliente asigna un número de puerto local a la máquina en la cual está siendo ejecutado. Ahora el cliente y el servidor pueden comunicarse escribiendo o leyendo en o desde sus respectivos sockets.

### 3.- JAVA Sockets

El paquete **java.net** de la plataforma Java proporciona una clase **Socket**, la cual implementa una de las partes de la comunicación bidireccional entre un programa Java y otro programa en la red.

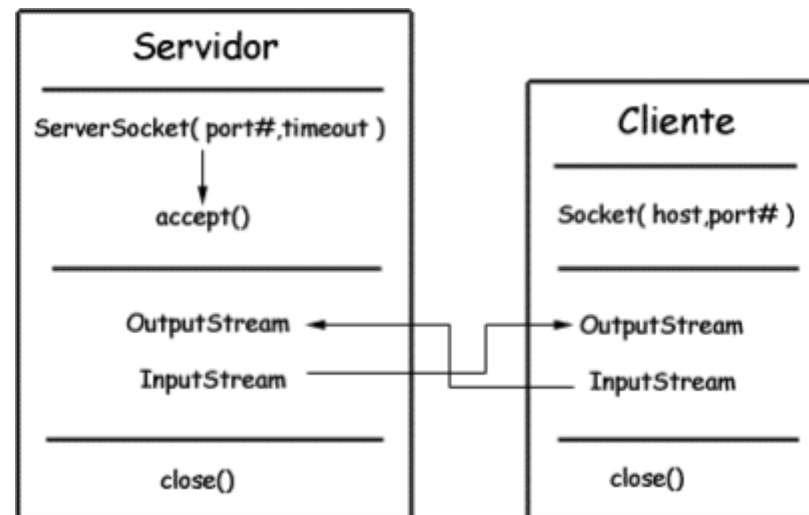
La clase **Socket** se sitúa en la parte más alta de una implementación dependiente de la plataforma, ocultando los detalles de cualquier sistema particular al programa Java. Usando la clase **java.net.Socket** en lugar de utilizar código nativo de la plataforma, los programas Java pueden comunicarse a través de la red de una forma totalmente independiente de la plataforma.

De forma adicional, **java.net** incluye la clase **ServerSocket**, la cual implementa un socket el cual los servidores pueden utilizar para escuchar y aceptar peticiones de conexión de clientes.

### 3.2.- Modelo de comunicaciones con Java

El modelo de sockets más simple es:

- El servidor establece un puerto y espera durante un cierto tiempo (*timeout* segundos), a que el cliente establezca la conexión. Cuando el cliente solicite una conexión, el servidor abrirá la conexión *socket* con el método *accept()*.
- El cliente establece una conexión con la máquina host a través del puerto que se designe en *puerto#*
- El cliente y el servidor se comunican con manejadores **InputStream** y **OutputStream**



### 3.3.- Apertura de Sockets

Si estamos programando un **CLIENTE**, el socket se abre de la forma:

Socket miCliente;

miCliente = new Socket( "maquina", numeroPuerto );

Donde *maquina* es el nombre de la máquina en donde estamos intentando abrir la conexión y *numeroPuerto* es el puerto (un número) del servidor que está corriendo sobre el cual nos queremos conectar.

Cuando se selecciona un número de puerto, se debe tener en cuenta que los puertos en el rango 0-1023 están reservados para usuarios con muchos privilegios (superusuarios o root). Estos puertos son los que utilizan los servicios estándar del sistema como email, ftp o http. Para las aplicaciones que se desarrollen, asegurarse de seleccionar un puerto por encima del 1023.

En el ejemplo anterior no se usan excepciones; sin embargo, es una gran idea la captura de excepciones cuando se está trabajando con sockets. El mismo ejemplo quedaría como:

```
Socket miCliente;  
try {  
    miCliente = new Socket( "maquina",numeroPuerto );  
} catch( IOException e ) {  
    System.out.println( e );  
}
```

Si estamos programando un **SERVIDOR**, la forma de apertura del *socket* es la que muestra el siguiente ejemplo:

```
Socket miServicio;  
try {  
    miServicio = new ServerSocket( numeroPuerto );  
} catch( IOException e ) {  
    System.out.println( e );  
}
```

A la hora de la implementación de un servidor también necesitamos crear un objeto *socket* desde el **ServerSocket** para que esté atento a las conexiones que le puedan realizar clientes potenciales y poder aceptar esas conexiones:

```
Socket socketServicio = null;  
try {  
    socketServicio = miServicio.accept();  
} catch( IOException e ) {  
    System.out.println( e );  
}
```